Qwen

Maturidade Técnica em Tempo Limitado: Arquitetura e Segurança para MVP de Caronas com Supabase

Estratégia de Autenticação e Validação Institucional

A criação de um sistema de caronas exclusivo para estudantes exige uma camada inicial de segurança e credenciamento rigorosa para garantir a integridade da comunidade. A restrição de registro a domínios de e-mail institucionais, como `@fatec.sp.gov.br`, é um requisito central que serve como a primeira linha de defesa contra o acesso não autorizado. A implementação dessa validação não pode ser deixada ao acaso; ela deve ser integrada diretamente na infraestrutura de banco de dados para ser intransponível. A análise das melhores práticas e funcionalidades do Supabase revela uma abordagem superior para esta tarefa, demonstrando maturidade técnica ao priorizar a segurança no nível mais baixo possível. A solução proposta utiliza um gatilho PostgreSQL com privilégios elevados para interceptar cada tentativa de inscrição de novo usuário, validando o domínio do e-mail antes mesmo que ele seja permanentemente gravado no sistema. Esta abordagem contrasta com métodos de camada de aplicação, que podem ser contornados ou subestimados. Para gerenciar dinamicamente a lista de domínios permitidos, será criada uma tabela dedicada, `public.allowed_email_domains`, que armazenará os domínios aprovados [6]. Essa tabela não só permite a gestão sem a necessidade de alterar o código-fonte, mas também pode ter sua própria política de segurança em nível de linha (RLS) para restringir quem pode adicionar ou remover domínios da lista.

O mecanismo principal para a validação é uma função PL/pgSQL marcada com `SECURITY DEFINER`. O atributo `SECURITY DEFINER` é crucial, pois faz com que a função seja executada com os privilégios de seu criador, geralmente o superusuário `postgres`, em vez dos privilégios do usuário que chamou a função [6]. Isso é indispensável neste cenário, pois o trigger dispara antes que o novo registro de usuário exista no sistema, tornando impossível acessar o JWT ou outras informações do usuário autenticado diretamente. A função recebe o novo usuário a ser inserido (`NEW`) e extrai o domínio do seu endereço de e-mail, convertendo-o para minúsculas para garantir a normalização [6]. Em seguida, ela consulta a tabela

`public.allowed_email_domains` para verificar se esse domínio existe na lista. Se o domínio não for encontrado, a função interrompe o processo de inserção lançando uma exceção personalizada, como "Signups from the domain % are not allowed" [6]. Essa exceção é capturada pelo PostgreSQL, que aborta a transação e impede que o novo usuário seja registrado. O trigger é então anexado à tabela `auth.users` e acionado antes de cada inserção (`BEFORE INSERT ON auth.users`), garantindo que a validação ocorra sistematicamente para todas as novas inscrições, independentemente do método utilizado (email/senha, OAuth, link mágico) [6] [11].

Como alternativa, o Supabase oferece um mecanismo mais moderno baseado em webhooks através do hook `before-user-created` [7]. Este hook permite que uma função de servidor (como uma Função Deno Edge) seja invocada antes que um novo usuário seja efetivamente criado no banco de dados. O payload enviado para essa função contém as informações do usuário, incluindo o e-mail, permitindo que a lógica de validação seja executada fora do banco de dados. Se a validação falhar, a função do webhook pode retornar uma resposta de erro HTTP (por exemplo, status 403 Proibido), abortando assim o processo de inscrição [7]. Embora esta abordagem ofereça flexibilidade, a solução via trigger PostgreSQL permanece como a escolha mais robusta e segura para um contexto acadêmico onde a demonstração de controle sobre a integridade do banco de dados é valorizada. A validação direta no nível do banco de dados elimina dependências externas e garante que a regra de negócio esteja imutavelmente codificada na estrutura de dados.

Para armazenar uma flag de verificação após a aprovação, a melhor prática é criar uma tabela de perfil separada, por exemplo, `public.profiles`, que se relaciona com a tabela `auth.users` via chave estrangeira [4]. Esta tabela pode conter metadados adicionais do usuário, como nomes e, crucialmente, uma coluna booleana como `is_institution_verified`. Quando um novo usuário se inscreve com sucesso, um disparador na tabela `auth.users` pode automaticamente criar um registro correspondente na tabela `public.profiles` com essa flag definida como verdadeira [4] [5]. Para garantir que essa flag seja definida corretamente, a tabela `profiles` deve ter RLS habilitado e políticas apropriadas para permitir que o próprio usuário crie e atualize seu perfil. Uma política de inserção poderia ser `CREATE POLICY "Users can create their own profile." ON profiles FOR INSERT TO authenticated WITH CHECK (auth.uid() = user_id);` [1]. Isso assegura que a criação do perfil seja atrelada à identidade confirmada do usuário. A combinação da validação de domínio via gatilho e o armazenamento de metadados no perfil demonstra uma compreensão completa do ciclo de vida do usuário, desde a entrada até a representação de seus dados dentro da aplicação.

Componente	Descrição	Justificativa Técnica
Tabela de Domínios Permitidos	`public.allowed_email_domains` com colunas `id`, `domain` (único) e `created_at`.	Permite uma gestão dinâmica da lista de domínios sem modificar código. A unicidade evita duplicatas. [6]
Função de Trigger	Função PL/pgSQL com `SECURITY DEFINER` que verifica o domínio do e-mail contra a tabela de domínios permitidos.	Executa com privilégios elevados para acessar a tabela de domínios antes do usuário ser criado. Bane o registro se o domínio não for encontrado. [6]
Trigger de Inscrição	Gatilho `BEFORE INSERT` anexado à tabela `auth.users` que executa a função de validação.	Garante que a regra de validação seja aplicada a todas as novas inscrições, independentemente do método de autenticação. [6] [11]
Tabela de Perfis	`public.profiles` com `user_id` como chave primária e estrangeira para `auth.users`. Inclui a flag `is_verified_institutionally`.	Centraliza os metadados do usuário e permite a criação de políticas de segurança granulares. [4] [5]

Em resumo, a estratégia de validação institucional proposta é um pilar fundamental para a segurança e credibilidade do sistema. Ao utilizar triggers PostgreSQL com `SECURITY DEFINER`, a regra de negócio é solidificada no banco de dados, criando uma barreira de segurança intransponível. A gestão dinâmica através de uma tabela dedicada e a sincronização automática de metadados para uma tabela de perfis demonstram um entendimento profundo das capacidades do Supabase e dos princípios de engenharia de software, posicionando a solução tecnicamente para uma apresentação em Ideathon.

Arquitetura de Software e Documentação Visual

A construção de um sistema robusto e escalável começa com uma arquitetura bem definida e comunicada. Para o MVP de caronas estudantis, uma arquitetura híbrida que combina um cliente frontend moderno, um servidor backend intermediário e um conjunto de serviços de banco de dados gerenciados (Supabase) oferece o equilíbrio ideal entre velocidade de desenvolvimento e controle de segurança. Esta arquitetura permite delegar tarefas complexas de segurança e gerenciamento de dados para o Supabase, enquanto o backend Node.js foca em lógica de negócios específica da aplicação e na gestão segura de sessões. A documentação visual através de Diagramas de Componentes e Diagramas de Sequência é essencial para traduzir essa arquitetura em um modelo compreensível, demonstrando uma visão holística do sistema tanto em estado estático quanto dinâmico. Esses diagramas servem como um documento técnico preciso que alinha a equipe e comunica a estrutura da solução para avaliadores.

O **Diagrama de Componentes** mapeia a arquitetura estática do sistema, destacando os blocos de construção principais e as interfaces pelas quais eles se comunicam. Os três componentes centrais são: 1. **Componente Frontend:** Representa a interface do usuário, que pode ser construída com tecnologias como React, Vue ou Angular, ou um framework SSR como Next.js [19]. Este componente é responsável por renderizar a UI e interagir com o usuário. Ele não tem conhecimento direto do banco de dados PostgreSQL. Em vez disso, ele se comunica com o mundo externo através da API REST exposta pelo Supabase PostgREST [15]. 2. **Componente Backend (Node.js/Express.js):** Este é um serviço de API intermediário, construído com Node.js e o framework Express.js. Suas responsabilidades incluem: receber requisições do frontend, gerenciar sessões de forma segura (idealmente usando cookies com a opção `httpOnly` para mitigar ataques XSS [8]), implementar lógica de negócio customizada que não se encaixa bem em políticas de segurança em nível de linha (RLS), e agir como um proxy seguro para o banco de dados. O backend Express.js não deve usar a chave `service_role` do Supabase para acessar dados de usuários, pois isso burlaria completamente o RLS [1] [9]. Em vez disso, ele deve atuar como um cliente confiável, recebendo o token JWT do usuário e utilizando-o para fazer chamadas autorizadas à API do Supabase. 3. **Componente Supabase Services:** Este componente representa a suite de serviços do Supabase, que funciona como o núcleo de dados e autenticação da aplicação. Ele se divide em várias sub-rotinas:

- **GoTrue:** Gerencia a autenticação de usuários, emitindo tokens JWT (JSON Web Tokens) após a verificação de credenciais [15] [16].
- **PostgreSQL Database:** O banco de dados relacional central, onde todas as informações da aplicação (usuários, caronas, etc.) são armazenadas [15].
- **PostgREST:** Transforma automaticamente o esquema do PostgreSQL em uma API RESTful, permitindo que o backend (e o frontend, em casos menos críticos) faça requisições HTTP para ler e escrever dados [15].
- **Row Level Security (RLS):** Um mecanismo de segurança embutido no PostgreSQL que aplica filtros de linha em tempo de execução para garantir que os usuários só possam acessar os dados de sua propriedade [2] [14].

A comunicação entre os componentes ocorre principalmente através de APIs REST. O frontend se comunica com o backend via HTTPS. O backend, por sua vez, se comunica com o Supabase (PostgREST) via HTTPS, incluindo o token JWT do usuário no cabeçalho de autorização para que o RLS seja ativado [9].

O **Diagrama de Sequência**, por outro lado, modela a arquitetura dinâmica, focando em um caso de uso específico: a solicitação de uma carona. Este diagrama detalha a troca de

mensagens entre os objetos do sistema ao longo do tempo para cumprir essa funcionalidade. A seguir, um trecho detalhado do fluxo:

1. **Início do Fluxo:** O usuário, já autenticado no sistema, interage com a interface do frontend (por exemplo, clica em um botão "Solicitar Carona"). O frontend coleta as informações necessárias (origem, destino, data/horário).
2. **Requisição (Frontend -> Backend):** O frontend envia uma requisição HTTP `POST` para o endpoint `/api/rides/request` no servidor Node.js. A requisição inclui os dados da carona no corpo e, crucialmente, o token JWT de acesso do usuário no cabeçalho de autorização (ex: `Authorization: Bearer <token>`).
3. **Middleware de Autenticação (Backend):** A rota `/api/rides/request` no Express.js está protegida por um middleware de autenticação. Este middleware é a primeira linha de defesa na camada de aplicação. Ele extrai o token do cabeçalho, valida sua assinatura pública usando o JWKS (JSON Web Key Set) do Supabase, e verifica as claims padrão como o emissor (`iss`) e o público-alvo (`aud`) [9]. Este passo complementa a segurança do RLS, agindo como uma camada de defesa em profundidade [2] [20]. Se a validação falhar, o middleware retorna um erro `401 Unauthorized` e o fluxo é encerrado.
4. **Persistência de Dados (Backend -> Supabase DB):** Com o token validado, o middleware extraí o ID do usuário (`sub` ou `uid` do JWT) e passa a requisição para o próximo controlador. Este controlador usa um cliente Supabase (configurado com a chave `anon` ou outra chave de baixo privilégio) para fazer uma chamada à API REST do PostgREST. Ele realiza uma requisição `POST` para o endpoint `/rides` (que corresponde à tabela `public.rides`), injetando o mesmo token JWT no cabeçalho de autorização. Dentro do PostgreSQL, o RLS está habilitado nesta tabela e uma política `INSERT` irá filtrar a inserção, garantindo que o `driver_id` ou `passenger_id` inserido corresponda ao `auth.uid()` obtido do token [1] [22].
5. **Confirmação e Resposta (DB -> Backend -> Frontend):** Se a inserção no banco de dados for bem-sucedida, o Supabase retorna uma resposta de sucesso (ex: `201 Created`) para o backend. O backend, por sua vez, processa essa resposta e a repassa ao frontend com uma mensagem de sucesso, como "Solicitação de carona enviada com sucesso".
6. **Atualização da Interface (Frontend):** O frontend, recebendo a resposta de sucesso, atualiza a interface do usuário para refletir o status da solicitação, talvez removendo o formulário ou exibindo uma mensagem de confirmação.

Este fluxo ilustra uma arquitetura bem projetada onde cada componente tem uma responsabilidade clara. O backend não apenas age como um proxy, mas também como um validador de segurança, reforçando a política de segurança estabelecida no banco de dados. A inclusão deste diagrama de sequência no relatório demonstra uma compreensão

aprofundada dos ciclos de vida de requisição e da importância de múltiplas camadas de segurança.

Fundamentos da Segurança com Row Level Security (RLS)

A segurança de dados é o pilar mais crítico de qualquer aplicação que lida com informações pessoais, e para um MVP de caronas estudantis, a proteção dos dados de usuários (motoristas, passageiros, localizações, histórico de corridas) é de suma importância. O Supabase, por sua natureza, expõe o banco de dados PostgreSQL através de uma API RESTful gerada pelo PostgREST, o que, se não for devidamente configurado, representa um risco de segurança significativo [14] [21]. Qualquer pessoa com o `anon` key (chave anônima) publicável poderia, por padrão, ler, inserir, atualizar ou excluir dados em qualquer tabela do esquema público [10] [22]. É aqui que a Segurança em Nível de Linha (RLS, na sigla em inglês) entra como a ferramenta de segurança primária e indispensável. A RLS é um recurso nativo do PostgreSQL que permite aos desenvolvedores definir políticas de acesso que filtram linhas de dados com base no contexto da sessão do usuário [2] [14]. Ela funciona como uma "camada de segurança" (security floor), garantindo que, independentemente de como uma consulta chega ao banco de dados — seja via SDK do cliente, chamada de API REST direta ou conexão direta — o resultado será sempre restrito pelas políticas definidas [2] [13].

O mecanismo de segurança do Supabase é intrinsecamente ligado ao seu sistema de autenticação. Quando um usuário se autentica, o Supabase GoTrue emite um Token Web JSON (JWT). Esse token contém informações sobre o usuário, incluindo seu ID único (`uid`). Quando uma requisição de API para o PostgREST é feita com esse JWT, o serviço automaticamente decodifica o token e o disponibiliza ao PostgreSQL através de funções auxiliares fornecidas pelo Supabase, como `auth.uid()` e `auth.role()` [14] [16]. As políticas RLS utilizam precisamente essas funções para tomar decisões de acesso. Por exemplo, uma política pode dizer: "para esta consulta `SELECT`, retorne apenas as linhas onde o valor na coluna `user_id` é igual ao valor retornado pela função `auth.uid()`" [1]. A RLS automaticamente adiciona essa condição como uma cláusula `WHERE` a todas as consultas na tabela, tornando a filtragem transparente para a aplicação [10].

A implementação correta da RLS segue um fluxo de trabalho consistente para cada tabela que contém dados sensíveis ou proprietários: 1. **Habilitar a RLS:** A primeira e mais

crítica etapa é habilitar explicitamente a RLS na tabela. Isso é feito com o comando SQL `ALTER TABLE nome_da_tabela ENABLE ROW LEVEL SECURITY;` [13] [14]. Se a RLS não for habilitada, a tabela permanecerá vulnerável, mesmo que políticas sejam criadas. 2. **Analisar o Escopo de Acesso:** Antes de escrever políticas, é necessário definir claramente quem pode fazer o quê. Para a maioria das entidades de usuário (perfis, caronas, avaliações), a regra de acesso mais comum é a de "apenas os próprios dados", onde cada usuário possui e controla seus próprios registros. 3. **Criar Políticas Granulares:** Baseado na análise, são criadas políticas específicas para cada operação (SELECT, INSERT, UPDATE, DELETE) e para cada papel de usuário (geralmente `authenticated` para usuários logados e `anon` para visitantes não logados) [1]. É uma prática recomendada criar quatro políticas distintas para cobrir todo o ciclo de vida CRUD (Create, Read, Update, Delete) de um registro pertencente a um usuário [11] [13].

Um dos conceitos mais importantes e frequentemente mal compreendidos na RLS é a distinção entre as cláusulas `USING` e `WITH CHECK` [2] [12]. Essa distinção é fundamental para prevenir vazamentos de segurança e inconsistências de dados.

- **Cláusula `USING`:** Esta cláusula é usada para operações de leitura e para selecionar quais linhas serão afetadas por uma operação de atualização ou exclusão. Ela define um filtro para a cláusula `WHERE` de uma consulta. Por exemplo, `USING (auth.uid() = user_id)` significa "permita que o usuário veja, atualize ou exclua apenas as linhas onde o `user_id` da linha corresponde ao ID do usuário autenticado" [1] [11].
- **Cláusula `WITH CHECK`:** Esta cláusula é usada exclusivamente para operações de escrita (inserção e atualização). Ela valida o *novo* conjunto de dados que está sendo inserido ou modificado. Ela define um filtro para garantir que o novo registro "passe no teste" de autorização. Por exemplo, `WITH CHECK (auth.uid() = user_id)` significa "permita que o usuário insira um novo registro apenas se o `user_id` do novo registro corresponder ao ID do usuário autenticado" [1] [2].

A combinação dessas duas cláusulas é vital para a operação de atualização. Uma política de `UPDATE` precisa de ambas para ser segura: `USING` para garantir que o usuário possa selecionar a linha que deseja atualizar, e `WITH CHECK` para garantir que, mesmo que o usuário tente modificar a coluna de propriedade (ex: mudar o `user_id` de uma de suas postagens para o ID de outra pessoa), a operação seja rejeitada [2] [11]. Omissões comuns, como esquecer a cláusula `WITH CHECK` em políticas de inserção e atualização, podem levar a vulnerabilidades graves onde um usuário pode "roubar" a posse de registros pertencentes a outros [2] [22].

A RLS deve ser vista como parte de uma estratégia de segurança em profundidade ("defense-in-depth"), onde ela funciona como a última linha de defesa, garantindo a integridade dos dados mesmo que haja falhas na camada de aplicação [2] [20]. No entanto, ela não substitui a necessidade de lógica de validação no backend. O uso combinado de RLS no banco de dados e middlewares de validação no servidor backend cria uma arquitetura muito mais resiliente a ataques e erros de programação [2] [13]. Além disso, é importante entender que, por padrão, o RLS falha silenciosamente para operações de seleção, atualização e exclusão, retornando zero resultados em vez de lançar um erro [10]. Isso significa que testes automatizados são absolutamente essenciais para garantir que as políticas estejam funcionando conforme o esperado, verificando explicitamente a ausência de resultados quando um usuário tenta acessar dados alheios [10] [11].

Configuração Prioritária de Políticas RLS para Proteção de Dados

A configuração das políticas de Segurança em Nível de Linha (RLS) é o núcleo técnico da segurança do sistema de caronas estudantis. Esta seção detalha a implementação prioritária dessas políticas para as tabelas mais críticas do banco de dados, garantindo que os dados dos usuários sejam estritamente protegidos sob a regra de "apenas os próprios dados". A estratégia adotada é a de criar um conjunto completo de quatro políticas (uma para cada operação de `SELECT`, `INSERT`, `UPDATE` e `DELETE`) para cada tabela que contém dados proprietários, como `public.profiles` e `public.ride_requests` [11] [13]. A implementação é realizada diretamente no SQL Editor do Dashboard do Supabase, uma vez que estas são configurações fundamentais do banco de dados.

Primeiramente, é crucial habilitar a RLS em cada tabela que requer proteção. Isso é feito com um simples comando `ALTER TABLE`:

```
-- Habilita a RLS na tabela de perfis
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

-- Habilita a RLS na tabela de solicitações de carona
ALTER TABLE public.ride_requests ENABLE ROW LEVEL SECURITY;
```

Após habilitar a RLS, a tabela fica inacessível para consultas `SELECT` de usuários autenticados até que uma política de `USING` seja criada, pois o comportamento padrão ao habilitar a RLS sem políticas é retornar zero linhas [13] [22].

A seguir, são apresentadas as políticas RLS prioritárias para as tabelas-chave, juntamente com uma explicação detalhada de cada uma.

1. Política para a Tabela `public.profiles`

Esta tabela armazena os metadados básicos do usuário (nome, foto, etc.) e é um alvo primário para a proteção de dados.

- **Política de Seleção (`SELECT`):** Permite que um usuário autenticado visualize seu próprio perfil.

```sql
CREATE POLICY "Users can view their own profile."
ON public.profiles
FOR SELECT
TO authenticated
USING ( auth.uid() = user_id );
```

Esta política diz: "Para operações de `SELEÇÃO`, permita que o papel `aute

- **Política de Inserção (`INSERT`):** Permite que um usuário autenticado crie seu próprio registro de perfil. A cláusula `COM VERIFICAÇÃO` é usada para garantir que o usuário só possa inserir um perfil que lhe pertença.

```sql
CREATE POLICY "Users can create their own profile."
ON public.profiles
FOR INSERT
TO authenticated
WITH CHECK ( auth.uid() = user_id );
```

Aqui, `COM VERIFICAÇÃO` garante que o `user_id` do novo registro inserido

- **Política de Atualização (`UPDATE`):** Permite que um usuário autenticado atualize seu próprio perfil. Requer ambas as cláusulas `USING` e `WITH CHECK` para segurança máxima. `USING` garante que o usuário possa selecionar seu próprio perfil para edição, e `WITH CHECK` garante que ele não possa alterar a propriedade do perfil.

```sql
CREATE POLICY "Users can update their own profile."
ON public.profiles
FOR UPDATE
TO authenticated
USING ( auth.uid() = user_id )
WITH CHECK ( auth.uid() = user_id );
```
Ambas as cláusulas são idênticas neste caso, pois estamos apenas verifican

- **Política de Exclusão (`DELETE`):** Permite que um usuário autenticado exclua seu próprio perfil. A cláusula `USING` é suficiente aqui, pois não há dados novos para verificar.

```sql
CREATE POLICY "Users can delete their own profile."
ON public.profiles
FOR DELETE
TO authenticated
USING ( auth.uid() = user_id );
```
A política permite que o usuário exclua a linha apenas se ela for dele <su

2. Política para a Tabela `public.ride_requests`

Esta tabela armazena as solicitações de carona feitas pelos usuários.

- **Política de Seleção (`SELECT`):** Permite que um usuário veja suas próprias solicitações de carona.

```sql
CREATE POLICY "Passengers can view their own ride requests."
ON public.ride_requests
FOR SELECT
TO authenticated
USING ( auth.uid() = passenger_id );
```
Assumindo que a tabela `ride_requests` tenha uma coluna `passenger_id` que

- **Política de Inserção (`INSERT`):** Permite que um usuário crie uma nova solicitação de carona.

```sql
CREATE POLICY "Passengers can create their own ride request."
ON public.ride_requests
FOR INSERT
TO authenticated
WITH CHECK ( auth.uid() = passenger_id );
```

- **Política de Atualização (`UPDATE`):** Permite que um usuário atualize seu próprio pedido de carona (ex: cancelar).

```sql
CREATE POLICY "Passengers can update their own ride request."
ON public.ride_requests
FOR UPDATE
TO authenticated
USING ( auth.uid() = passenger_id )
WITH CHECK ( auth.uid() = passenger_id );
```

- **Política de Exclusão (`DELETE`):** Permite que um usuário cancele sua própria solicitação.

```sql
CREATE POLICY "Passengers can delete their own ride request."
ON public.ride_requests
FOR DELETE
TO authenticated
USING ( auth.uid() = passenger_id );
```

Além das políticas de "proprietário", pode-se considerar políticas públicas para operações de leitura. Por exemplo, para permitir que todos vejam uma lista de caronas disponíveis (sem informações de contato sensíveis), uma política pública pode ser criada:

```
CREATE POLICY "Public can see available rides."
ON public.ride_requests
FOR SELECT
TO anon, authenticated
USING ( status = 'AVAILABLE' AND sensitive_data IS NULL );
```

Esta política permite que tanto usuários anônimos quanto autenticados leiam as linhas onde a carona está disponível, mas somente se os campos de dados sensíveis forem nulos [22].

Desempenho e Indexação: Uma falha catastrófica na implementação de RLS é negligenciar o desempenho. Sem índices adequados, as políticas podem transformar consultas rápidas em lentas varreduras sequenciais de toda a tabela, tornando a aplicação inutilizável [2] [12]. Para cada coluna usada em uma expressão de política (como `user_id`, `passenger_id`), deve-se criar um índice B-tree. Por exemplo:

```
-- Índice para a política de perfis
CREATE INDEX idx_profiles_user_id ON public.profiles (user_id);

-- Índice para a política de solicitações de carona
CREATE INDEX idx_ride_requests_passenger_id ON public.ride_requests (passe
```

Estes índices permitem que o PostgreSQL localize rapidamente as linhas relevantes para a avaliação da política, em vez de verificar cada linha da tabela [1] [2].

Outra otimização de desempenho crucial é envolver a chamada de `auth.uid()` em um subselect `(SELECT auth.uid())`. Isso instrui o planejador do PostgreSQL a executar a função uma única vez e cachear o resultado, em vez de chamá-la para cada linha durante a verificação da política, resultando em ganhos de performance drásticos [10] [12] [13].

A tabela abaixo resume as políticas prioritárias para as tabelas `profiles` e `ride_requests`.

Tabela	Operação	Papel	Condição (`USING` / `WITH CHECK`)	Propósito
`public.profiles`	`SELECT`	`authenticated`	`auth.uid() = user_id`	Usuário vê seu próprio perfil.
`public.profiles`	`INSERT`	`authenticated`	`auth.uid() = user_id`	Usuário cria seu próprio perfil.
`public.profiles`	`UPDATE`	`authenticated`	`USING`: `auth.uid() = user_id` `WITH CHECK`: `auth.uid() = user_id`	Usuário atualiza seu próprio perfil.
`public.profiles`	`DELETE`	`authenticated`	`auth.uid() = user_id`	Usuário exclui seu próprio perfil.
`public.ride_requests`	`SELECT`	`authenticated`	`auth.uid() = passenger_id`	Passageiro vê suas próprias solicitações.
`public.ride_requests`	`INSERT`	`authenticated`	`auth.uid() = passenger_id`	Passageiro cria uma nova solicitação.
`public.ride_requests`	`UPDATE`	`authenticated`	`USING`: `auth.uid() = passenger_id` `WITH CHECK`: `auth.uid() = passenger_id`	Passageiro atualiza sua solicitação.
`public.ride_requests`	`DELETE`	`authenticated`	`auth.uid() = passenger_id`	Passageiro cancela sua solicitação.

Ao implementar estas políticas e otimizações, a aplicação se beneficia de uma proteção de dados robusta e performática, onde a segurança é garantida diretamente pelo banco de dados, permitindo que a camada de aplicação se concentre na lógica de negócios.

Defesa em Profundidade: Middlewares e Práticas Complementares

Embora a Segurança em Nível de Linha (RLS) seja o pilar da segurança de dados no Supabase, uma estratégia de defesa em profundidade reconhece que nenhum único mecanismo de segurança é infalível [2] [20]. A combinação de múltiplas camadas de defesa, onde a falha de uma camada é compensada por outra, resulta em uma arquitetura muito mais resiliente. Neste contexto, o backend Node.js e seus middlewares desempenham um papel crucial como uma segunda linha de defesa, complementando a segurança imutável do RLS. Esta abordagem demonstra um entendimento sofisticado de arquitetura de segurança, indo além da simples configuração de políticas de banco de dados.

A principal função do middleware de autenticação no Express.js é validar o JWT recebido da requisição do cliente antes que qualquer lógica de negócios seja executada [9]. Este

middleware deve realizar várias verificações: 1. **Extrair o Token:** Extrair o token do cabeçalho de autorização da requisição. 2. **Verificar Assinatura:** Decodificar e verificar a assinatura do JWT usando as chaves públicas do Supabase (disponíveis no JWKS - JSON Web Key Set) [9]. Isso garante que o token não foi adulterado. 3. **Validar Claims Padrão:** Verificar as claims padrão do JWT, como o issuer (`iss`), que deve apontar para a URL correta do projeto Supabase, e o audience (`aud`), que deve corresponder à API do Supabase [9]. 4. **Verificar Expiração:** Validar a claim `exp` para garantir que o token não está vencido.

Essa validação local no backend é importante por vários motivos. Primeiro, ela filtra requisições malformadas ou fraudulentas logo no início, economizando recursos do servidor. Segundo, e mais importante, ela reforça a segurança do RLS. Mesmo que uma política de RLS falhasse por algum motivo inesperado, o fato de o backend já ter validado o token e extraído o `user_id` significa que a lógica subsequente (por exemplo, a chamada para o banco de dados) estará operando com base em uma identidade de usuário já verificada. Isso cria uma redundância de segurança, onde a identidade do usuário é validada tanto no cliente (indiretamente, através da presença do token) quanto no servidor, e o acesso aos dados é finalmente filtrado no banco de dados [2] [13].

Uma decisão técnica crítica no backend é como ele se autentica junto ao Supabase. O Supabase fornece duas chaves principais: a `SUPABASE_ANON_KEY` (anônima) e a `SUPABASE_SERVICE_ROLE_KEY` (chave de função de serviço) [1] [11]. A `SUPABASE_ANON_KEY` deve ser usada em clientes e em chamadas de API que não exigem privilégios elevados, pois ela opera sob as políticas de RLS [22]. A `SUPABASE_SERVICE_ROLE_KEY`, por outro lado, concede acesso de superusuário que **ignora completamente todas as políticas de RLS** [1] [11]. Usar esta chave no backend para acessar dados de usuários seria uma grave violação de segurança, pois o backend passaria a ter poder total sobre o banco de dados, eliminando o principal mecanismo de segurança do Supabase.

A abordagem correta, conhecida como "Caminho A", é a seguinte: 1. O backend recebe o JWT do usuário autenticado a partir do frontend. 2. O backend valida o JWT localmente, como descrito anteriormente. 3. O backend então cria um cliente Supabase (usando um SDK como `@supabase/supabase-js`) e faz uma chamada à API do Supabase, injetando o JWT do usuário original no cabeçalho de autorização da requisição [9]. O cliente Supabase pode ser inicializado com a `SUPABASE_ANON_KEY`, mas a requisição individual deve especificar o token do usuário. Essa abordagem garante que o RLS seja ativado e execute suas políticas, mantendo a integridade da segurança em nível de linha [9].

Outra prática complementar é a gestão de permissões de coluna. Enquanto o RLS controla o acesso a linhas inteiras, ele não controla o acesso a colunas individuais [10]. Se uma tabela contiver colunas extremamente sensíveis (ex: um campo `tax_id` na tabela `profiles`), mesmo que um usuário seja bloqueado do ver ou editar outras linhas, ele ainda poderia potencialmente acessar essa coluna se o acesso `SELECT` à tabela fosse amplamente concedido. Para mitigar isso, as permissões de coluna no PostgreSQL devem ser gerenciadas explicitamente. Após revogar todos os privilégios padrão na tabela, os privilégios `SELECT`, `UPDATE` etc. devem ser concedidos explicitamente apenas às colunas necessárias para um determinado papel [10]. Por exemplo:

```
-- Revogar todos os privilégios da role 'authenticated'
REVOKE ALL ON TABLE public.profiles FROM authenticated;

-- Conceder permissão de UPDATE apenas às colunas 'first_name' e 'last_nam
GRANT UPDATE (first_name, last_name) ON TABLE public.profiles TO authentic
```

Isso adiciona uma camada de granularidade, limitando o que um usuário pode fazer até mesmo com as linhas que ele é permitido ver.

Finalmente, é imperativo abordar o cenário de falha silenciosa do RLS. Como mencionado, quando uma política `SELECT`, `UPDATE` ou `DELETE` não é satisfeita, o PostgreSQL simplesmente retorna zero resultados sem gerar um erro [10]. Isso pode mascarar problemas de política durante o desenvolvimento e a depuração. Portanto, a implementação de testes automatizados é indispensável. Esses testes devem simular tentativas de acesso de um usuário a dados pertencentes a outro e verificar que a resposta do banco de dados é, de fato, vazia. Ferramentas de teste de API como Postman ou bibliotecas de teste em JavaScript (como Mocha/Chai) podem ser usadas para este fim. O fluxo de teste típico envolve: 1. Registrar dois usuários de teste, `userA` e `userB`. 2. Fazer com que `userA` crie um recurso (ex: uma carona). 3. Fazer com que `userB` tente ler, atualizar ou excluir o recurso criado por `userA`. 4. Verificar que a resposta da API é `200 OK` (para `UPDATE`/`DELETE` que falham) ou `200 OK` com um corpo vazio (para `SELECT`), em vez de um erro de autorização ou o retorno dos dados de `userA` [10] [11].

Ao adotar uma abordagem de defesa em profundidade, combinando a força do RLS com validações robustas no backend, a gestão cuidadosa das chaves do Supabase, a aplicação de permissões de coluna e a implementação de testes automatizados, a aplicação atinge um nível de maturidade técnica e segurança que vai além do básico, demonstrando uma compreensão profunda dos desafios de segurança modernos.

Síntese e Recomendações para a Apresentação

Este relatório detalhou uma solução técnica completa e madura para um MVP de sistema de caronas estudantis, alinhada aos critérios exigidos para uma apresentação em um Ideathon de ADS. A proposta foi construída em torno de três pilares estratégicos: a validação institucional de usuários, a documentação arquitetural clara e a configuração prioritária de segurança com Row Level Security (RLS). Cada componente foi desenvolvido para demonstrar não apenas a viabilidade da execução rápida, mas também um profundo entendimento dos princípios de engenharia de software e segurança de dados.

A estratégia para a validação institucional de e-mails foi delineada com precisão, priorizando a abordagem via gatilho PostgreSQL com `SECURITY DEFINER` como a solução mais robusta e segura [6] [11]. Esta implementação solidifica a regra de negócio diretamente no banco de dados, criando uma barreira intransponível que demonstra maturidade técnica ao diferenciar a validação no nível da infraestrutura de soluções de camada de aplicação mais frágeis. A criação de uma tabela de domínios permitidos e a sincronização automática de flags de verificação em uma tabela de perfis completam o ciclo de vida do usuário, mostrando uma visão holística do sistema [4] [5].

A arquitetura proposta, documentada através de um Diagrama de Componentes e um trecho de Diagrama de Sequência, apresenta um modelo claro e modular. O Diagrama de Componentes destaca a separação de responsabilidades entre o Frontend, o Backend Node.js e os serviços Supabase, enquanto o Diagrama de Sequência detalha o fluxo dinâmico para a solicitação de uma carona. A inclusão do middleware de autenticação no Express.js como uma camada de validação JWT complementar é particularmente poderosa, pois ilustra a implementação de uma estratégia de defesa em profundidade, onde múltiplas camadas de segurança trabalham em conjunto para proteger a aplicação [2] [9] [20]. Esta visão abrangente mostra que a segurança não é um ponto isolado, mas um princípio que permeia toda a arquitetura.

O coração técnico da solução reside na configuração prioritária das Políticas de Segurança em Nível de Linha. Foram fornecidos blocos de código SQL exatos para as políticas de `SELECT`, `INSERT`, `UPDATE` e `DELETE` para as tabelas mais críticas, como `profiles` e `ride_requests`. A explicação detalhada da distinção fundamental entre as cláusulas `USING` e `WITH CHECK` demonstra um conhecimento sofisticado que vai além da simples cópia e colagem de código, mostrando a compreensão do "porquê" por trás de cada política [1] [2] [11]. Adicionalmente, as recomendações sobre otimizações de desempenho, como a criação de índices em colunas de política e o uso de subconsultas

para funções como `auth.uid()`, são essenciais para garantir que a aplicação seja segura e simultaneamente performática, evitando um dos erros mais comuns em sistemas RLS [10] [12] [13].

Em conclusão, a solução proposta atende e supera os requisitos do objetivo de pesquisa. Ela oferece uma base sólida para a apresentação no Ideathon, permitindo que o candidato demonstre uma maturidade técnica evidente. A abordagem equilibra a execução rápida exigida por um evento competitivo com uma arquitetura robusta e fundamentada em boas práticas de segurança e engenharia de software. Ao seguir este plano, o candidato estará preparado para justificar suas decisões técnicas, responder a perguntas aprofundadas da banca e, consequentemente, apresentar uma solução de alta qualidade e impacto.

Referência

1. Row Level Security | Supabase Docs https://supabase.com/docs/guides/database/postgres/row-level-security
2. Supabase RLS Guide 2026: Policies That Actually Work https://designrevision.com/blog/supabase-row-level-security
3. Supabase Custom Claims https://dev.to/supabase/supabase-custom-claims-34l2
4. User Management - Docs https://supabase.com/docs/guides/auth/managing-user-data
5. How to add additional metadata to users table? #6363 https://github.com/orgs/supabase/discussions/6363
6. Restrict Signups by Email Domain in Supabase https://www.rapidevelopers.com/supabase-tutorial/how-to-allow-sign-in-only-with-specific-domains-in-supabase
7. Before User Created Hook | Supabase Docs https://supabase.com/docs/guides/auth/auth-hooks/before-user-created-hook
8. Building Authentication with Express.js and Supabase - Medium https://blog.devgenius.io/building-authentication-with-express-js-and-supabase-67731b287431
9. How to use Supabase Auth with a backend on Render https://render.com/articles/how-to-use-supabase-auth-with-a-backend-on-render

10. Supabase RLS Best Practices: Production Patterns for ... https://makerkit.dev/blog/tutorials/supabase-rls-best-practices
11. Supabase Row Level Security in Production: Patterns That ... https://dev.to/whoffagents/supabase-row-level-security-in-production-patterns-that-actually-work-2l78
12. RLS Performance and Best Practices https://supabase.com/kb/troubleshooting/rls-performance-and-best-practices-Z5Jjwv
13. Row Level Security in Supabase: Complete Guide for Next. ... https://blog.starmorph.com/blog/row-level-security-supabase-tables-nextjs
14. Demystifying Row Level Security (RLS) in Supabase with a ... https://www.arundhaj.com/blog/supabase-row-level-security-rls-explained.html
15. Architecture | Supabase Docs https://supabase.com/docs/guides/getting-started/architecture
16. Auth architecture | Supabase Docs https://supabase.com/docs/guides/auth/architecture
17. Crafting a Ride-Sharing Platform: A Journey Through ... https://medium.com/@tusharmalhotra_81114/crafting-a-ride-sharing-platform-a-journey-through-entity-and-service-extraction-ee15d800b34f
18. Sequence Diagram-matching a shared ride for a driver ... https://www.researchgate.net/figure/Sequence-Diagram-matching-a-shared-ride-for-a-driver-and-a-passenger_fig5_262566296
19. Do I need a separate API layer with Supabase + Next.js, or ... https://www.reddit.com/r/Supabase/comments/1qeq69w/do_i_need_a_separate_api_layer_with_supabase/
20. When should I use Row Level Security (RLS) instead of ... https://github.com/orgs/supabase/discussions/48665
21. How to secure Supabase Data API with Row Level Security https://www.facebook.com/groups/vibecodinglife/posts/1935220717066456/
22. How to Add Row-Level Security to a Supabase App in ... https://dev.to/aadesh-kumar/how-to-add-row-level-security-to-a-supabase-app-in-under-30-minutes-523i
23. How to Design a Database for Ride-Sharing and ... https://www.geeksforgeeks.org/sql/how-to-design-a-database-for-ride-sharing-and-carpooling-services/